

Universidade Federal do Amazonas (UFAM)

Centro de P&D em Tecnologia Eletrônica e da Informação (CETELI)

Programa de Pós-Graduação em Engenharia Elétrica — PPGE

Disciplina: Engenharia de Software de Tempo Real

Navegação Autônoma Ponto-a-Ponto com Desvio Reativo de Obstáculos para o Robô Móvel Khepera IV: uma Arquitetura Híbrida Baseada em Máquina de Estados Finitos em Ambiente de Simulação Webots

Matheus Serrão Uchôa*

Manaus — AM, Junho de 2026

Resumo

Este relatório apresenta o projeto, a implementação e a avaliação experimental de um controlador de navegação autônoma para o robô móvel de tração diferencial Khepera IV, com o objetivo de deslocá-lo de uma configuração inicial a uma configuração-alvo em um ambiente povoado por obstáculos, sem colisões, em condições representativas de competições de robótica. O trabalho foi conduzido no simulador Webots R2025a, utilizando o modelo oficial do Khepera IV. Propõe-se uma *arquitetura híbrida* que combina uma camada deliberativa mínima de atração ao alvo (controle proporcional de rumo a partir de localização por GPS e unidade inercial) com uma camada reativa de desvio de obstáculos inspirada nos campos potenciais e nos veículos de Braitenberg, arbitradas por uma *máquina de estados finitos* (NAVEGAR, GIRAR, CONTORNAR) que introduz *compromisso de contorno* no espírito dos algoritmos Bug, eliminando o aprisionamento em mínimos locais. Reporta-se, ainda, uma contribuição metodológica: a calibração *empírica* dos sensores infravermelhos por meio de telemetria instrumentada, que revelou uma discrepância significativa entre a leitura prevista em folha de dados (≈ 29) e a leitura medida em espaço livre (≈ 110), causada pela reflexão do piso. A análise dos registros de execução permitiu diagnosticar e corrigir duas falhas distintas (aprisionamento em mínimo local e rodopio por má calibração). Na configuração final, o robô percorreu a trajetória de forma livre de colisões, reduzindo a distância ao alvo de 2,83 m a menos de 0,07 m em 19,3 s de tempo simulado. Discute-se a relação do laço de controle com o modelo de tarefa periódica de sistemas de tempo real. Documenta-se, por fim, a *evolução para o robô físico*: substituição do GPS por odometria, *rastreamento de linha por ponto de controle virtual* (malha cinemática de controle em cascata) e uma camada deliberativa de *mapeamento com planejamento global A**.

Palavras-chave: robótica móvel; navegação autônoma; desvio de obstáculos; Khepera IV; campos potenciais; máquina de estados; sistemas de tempo real; Webots.

*Discente, PPGE/CETELI — Universidade Federal do Amazonas (UFAM), Manaus-AM, Brasil.

Sumário

1	Introdução	2
1.1	Definição do problema	2
1.2	Objetivos	3
1.3	Contribuições	3
1.4	Organização	3
2	Fundamentação Teórica e Revisão da Literatura	3
2.1	Deliberativo <i>versus</i> reativo: as duas escolas	3
2.2	Campos potenciais artificiais	3
2.3	Métodos completos e robustos ao ruído	4
2.4	A plataforma Khepera e o Khepera IV	4
2.5	Simulação e tempo real	4
3	Materiais e Métodos	4
3.1	Ambiente de simulação	4
3.2	Modelagem do ambiente	5
3.3	Sistema de localização	5
3.4	Arquitetura de controle proposta	5
3.4.1	Rodopio por má calibração	8
3.5	Resultado final	8
3.6	Discussão	10
4	Evolução para o Robô Físico Khepera IV	10
4.1	Odometria e o algoritmo Bug2 no robô real	10
4.2	Rastreamento de linha por ponto de controle virtual (ponto-P)	10
4.3	Camada deliberativa: mapa de ocupação e planejamento A^*	11
4.4	Geração automática de mapa e interface de operação	11
5	Conclusão e Trabalhos Futuros	12
A	Código-fonte completo do controlador	13

1 Introdução

A capacidade de um robô móvel deslocar-se de um ponto a outro de maneira segura e autônoma — evitando colisões — é um dos problemas fundamentais e mais estudados da robótica [14, 15]. Apesar de sua aparente simplicidade, o problema condensa questões profundas de percepção, representação, controle e tomada de decisão em tempo real, e serve de base para aplicações que vão de robôs de serviço e veículos autônomos a competições acadêmicas de robótica móvel.

Este trabalho aborda especificamente a tarefa de *navegação ponto-a-ponto com desvio de obstáculos* para o robô **Khepera IV** [12], plataforma de tração diferencial amplamente utilizada em pesquisa e ensino e disponível no acervo do CETELI/UFAM. Para permitir o desenvolvimento e a validação reproduzíveis do software de controle antes da transferência para o robô físico, adotou-se o simulador **Webots** [13], que disponibiliza um modelo fidedigno do Khepera IV.

1.1 Definição do problema

Seja $\mathbf{p} = (x, y)$ a posição do robô no plano e $\mathbf{p}_g = (x_g, y_g)$ a posição do alvo. Deseja-se uma política de controle que conduza $\mathbf{p} \rightarrow \mathbf{p}_g$ mantendo o robô no espaço livre $\mathcal{C}_{\text{free}}$ (sem interseção com obstáculos). A dificuldade central no caso do Khepera IV decorre de seu sensoramento de

proximidade ser de *curto alcance* (infravermelho útil entre aproximadamente 2 e 20 cm) e da *ausência de mapa* prévio do ambiente: o robô é, em essência, míope e desinformado, o que torra inviável o planejamento global clássico e privilegia estratégias reativas [5].

1.2 Objetivos

Objetivo geral: projetar, implementar e avaliar um controlador capaz de levar o Khepera IV de uma posição inicial a um alvo, desviando de todos os obstáculos do percurso.

Objetivos específicos:

- modelar um cenário de testes no Webots (arena, obstáculos e alvo);
- formular uma arquitetura de controle híbrida (deliberativa + reativa);
- garantir robustez frente ao problema de mínimos locais;
- calibrar empiricamente o sensoriamento por meio de telemetria; e
- validar quantitativamente o desempenho com base em dados de execução.

1.3 Contribuições

As principais contribuições deste relatório são: (i) uma arquitetura de controle híbrida arbitrada por máquina de estados finitos que une atração ao alvo, desvio reativo e *contorno comprometido* de obstáculos; (ii) uma metodologia de calibração empírica do sensoriamento infravermelho baseada em instrumentação e análise de telemetria, evidenciando o perigo de confiar cegamente em folhas de dados; e (iii) uma análise experimental, com dados reais de execução, do fenômeno de mínimo local e de sua mitigação.

1.4 Organização

A Seção 2 apresenta a fundamentação teórica e a revisão da literatura. A Seção 3 descreve materiais e métodos. A Seção ?? expõe e discute os resultados. A Seção 4 documenta a evolução para o robô físico (odometria, rastreamento por ponto de controle virtual e planejamento A^* sobre mapa). A Seção 5 conclui e aponta trabalhos futuros.

2 Fundamentação Teórica e Revisão da Literatura

2.1 Deliberativo *versus* reativo: as duas escolas

A história da navegação autônoma é marcada pela tensão entre dois paradigmas. O paradigma *deliberativo* (sentir–modelar–planejar–agir) foi inaugurado pelo robô *Shakey* [3], para o qual se concebeu o algoritmo de busca informada A^* [4]. Embora teoricamente elegante e *completo*, o paradigma exige um modelo de mundo acurado e é computacionalmente custoso, tornando-se frágil em ambientes dinâmicos ou pouco estruturados.

Em reação a essas limitações, o paradigma *reativo* (sentir–agir) propôs eliminar o modelo central. Suas raízes remontam às “tartarugas” eletromecânicas de Grey Walter [1], capazes de fototaxia e desvio de obstáculos com pouquíssimos componentes, e ganham formalização conceitual nos *veículos* de Braitenberg [2], nos quais o acoplamento direto sensor–motor já produz comportamentos complexos. A consolidação do paradigma deve-se à *arquitetura de subsunção* de Brooks [5, 6], que organiza o comportamento em camadas reativas com prioridade, dispensando representação explícita.

2.2 Campos potenciais artificiais

Khatib [7] propôs tratar a navegação como o movimento de uma partícula sob um campo de forças: o alvo gera uma força *atrativa* e cada obstáculo, uma força *repulsiva*, de modo que o

robô segue o gradiente do potencial resultante,

$$\vec{F}(\mathbf{p}) = \vec{F}_{\text{att}}(\mathbf{p}) + \sum_i \vec{F}_{\text{rep},i}(\mathbf{p}). \quad (1)$$

O método é computacionalmente barato e adequado a tempo real, mas possui uma fraqueza estrutural bem conhecida: *mínimos locais*, configurações em que a força atrativa e a repulsiva se anulam (tipicamente quando um obstáculo se interpõe entre robô e alvo), aprisionando o veículo. Esse fenômeno é central na análise experimental da Seção ??.

2.3 Métodos completos e robustos ao ruído

A garantia de completude com sensoramento mínimo é fornecida pelos *algoritmos Bug* [8]: ao encontrar um obstáculo, o robô compromete-se a *contorná-lo* (seguir sua fronteira) até poder retomar o curso ao alvo. Essa ideia de *compromisso de contorno* é precisamente o mecanismo adotado neste trabalho para escapar de mínimos locais. Para robustez frente a ruído e dinâmica, destacam-se ainda o *Vector Field Histogram* (VFH) [9] e o *Dynamic Window Approach* (DWA) [10].

2.4 A plataforma Khepera e o Khepera IV

A família Khepera, originada no LAMI/EPFL e comercializada pela K-Team [11], tornou-se referência em robótica móvel acadêmica por seu tamanho reduzido e rico sensoramento. O **Khepera IV** [12] é um robô de tração diferencial com cerca de 14 cm de diâmetro, dotado de 8 sensores infravermelhos de proximidade, 5 sensores ultrassônicos, câmera colorida e unidade de medição inercial (IMU), executando Linux embarcado. No presente trabalho utilizam-se os infravermelhos (desvio) e, em simulação, GPS e IMU (localização).

2.5 Simulação e tempo real

O Webots [13] é um simulador profissional de robótica que integra dinâmica de corpos rígidos, sensores e atuadores, e modelos validados de robôs comerciais, permitindo desenvolvimento de software *controller-in-the-loop*. Do ponto de vista de **sistemas de tempo real**, o laço de controle do robô é uma *tarefa periódica* de período fixo T : a cada ciclo, executam-se percepção, decisão e atuação, que devem caber dentro de T . Esse é exatamente o modelo de tarefa periódica de Liu e Layland [16], base da teoria de escalonabilidade em tempo real.

3 Materiais e Métodos

3.1 Ambiente de simulação

Utilizou-se o Webots R2025a em sistema operacional Windows 11. O controlador foi implementado em linguagem **C** (compilado pela cadeia de ferramentas MinGW-w64 embarcada no próprio Webots), escolha alinhada à natureza embarcada e de tempo real da disciplina. A Tabela 1 resume os dispositivos do Khepera IV efetivamente empregados.

Tabela 1: Dispositivos do Khepera IV utilizados no controlador.

Dispositivo	Identificador (Webots)	Função
Motor esquerdo	left wheel motor	atuação (velocidade)
Motor direito	right wheel motor	atuação (velocidade)
IR frontal	front infrared sensor	detecção de obstáculo
IR frontal-esq./dir.	front {left,right} infrared sensor	detecção lateral-frontal
IR esquerdo/direito	{left,right} infrared sensor	detecção lateral
GPS	gps	localização (x, y)
IMU	inertial unit	orientação (yaw)

3.2 Modelagem do ambiente

O cenário consiste em uma **RectangleArena** de 3×3 m (sistema de coordenadas ENU, eixo z vertical), com cinco caixas de madeira (**WoodenBox**) dispostas em *slalom* ao longo da diagonal, um marcador visual de alvo (disco vermelho) em $(1,0; 1,0)$ e o robô iniciando em $(-1,0; -1,0)$, orientado para o alvo. As dimensões e posições foram escolhidas de modo a (i) obstruir a rota direta, forçando manobras de desvio, e (ii) manter folgas navegáveis compatíveis com o diâmetro do robô e com o alcance curto dos infravermelhos.

3.3 Sistema de localização

A camada de atração ao alvo requer estimativa de pose. Em simulação, empregam-se o **GPS** (posição (x, y)) e a **IMU** (ângulo de guinada θ). Ressalva-se que o Khepera IV *físico* não dispõe de GPS; na transferência para o robô real, a estimativa de posição deve ser obtida por *odometria* (integração dos encoders de roda), sujeita a deriva acumulada [14], ou por um sistema de localização externo. Trata-se de uma simplificação metodológica deliberada, que isola o problema de navegação do problema de localização.

3.4 Arquitetura de controle proposta

A política de controle é uma **máquina de estados finitos** (FSM) híbrida com três estados, que arbitra entre a atração ao alvo e o desvio de obstáculos (Figura 1).

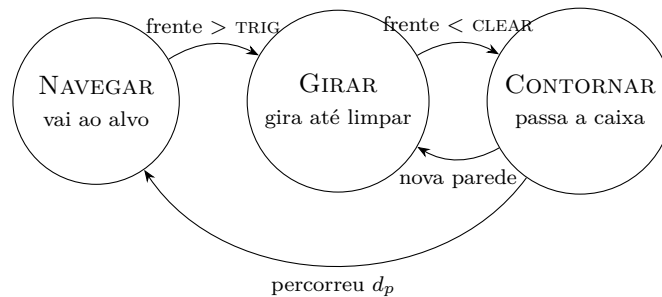


Figura 1: Máquina de estados finitos que arbitra atração ao alvo e desvio comprometido.

Percepção e atração (estado Navegar). A distância e o erro de rumo ao alvo são

$$d = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \quad (2)$$

$$e_\theta = \text{wrap}(\text{atan2}(y_g - y, x_g - x) - \theta) \in (-\pi, \pi], \quad (3)$$

em que $\text{wrap}(\cdot)$ normaliza o ângulo para evitar a “volta mais longa”. O comando de giro combina um termo proporcional de rumo (atração) com um termo reativo (repulsão):

$$\omega = \underbrace{K_\theta e_\theta}_{\text{atrativo}} + \underbrace{K_a (b_R - b_L)}_{\text{repulsivo}}, \quad (4)$$

onde as “massas” de obstáculo à esquerda e à direita são estimadas a partir das leituras infravermelhas $s_{(\cdot)}$ acima de uma linha de base s_0 :

$$b_L = \max(0, (s_{fl} - s_0) + \frac{1}{2}(s_l - s_0)), \quad (5)$$

$$b_R = \max(0, (s_{fr} - s_0) + \frac{1}{2}(s_r - s_0)). \quad (6)$$

Compromisso de contorno (estados Girar e Contornar). Quando a leitura frontal excede o limiar TRIG, a FSM transita para GIRAR: o robô *fixa* uma direção de escape (lado mais livre) e gira *no mesmo sentido* até que a frente esteja efetivamente livre ($< \text{CLEAR}$). Em seguida, em CONTORNAR, avança uma distância fixa d_p para ultrapassar o obstáculo antes de re-mirar o alvo. Esse *compromisso* — inspirado nos algoritmos Bug [8] — é o que rompe a indecisão característica do mínimo local (Seção ??).

Cinemática diferencial. O par (v, ω) é convertido em velocidades de roda. Na forma simplificada efetivamente utilizada (com ω já em unidades de roda),

$$\dot{\phi}_L = v - \omega, \quad \dot{\phi}_R = v + \omega, \quad (7)$$

que corresponde, a menos de constantes, à relação física $\dot{\phi}_{L,R} = \frac{1}{r}(v \mp \frac{\omega_z L}{2})$, sendo r o raio da roda e L a bitola.

Núcleo do controlador. A Listagem 1 reproduz o trecho central da FSM. O código completo encontra-se no Apêndice A.

```

1  switch (state) {
2    case NAVEGAR:
3      steer = K_HEADING*err + K_AVOID*(block_r - block_l);
4      forward = CRUISE;
5      forward *= (1.0 - 0.7*clampd((f-IR_BASE)/(IR_TRIG-IR_BASE),0,1));
6      forward *= clampd(1.0 - 0.5*fabs(err), 0.3, 1.0);
7      if (front > IR_TRIG) { /* parede a frente */
8        escape_dir = (block_l <= block_r) ? +1 : -1; /* lado mais livre */
9        state = GIRAR;
10     }
11     break;
12   case GIRAR: /* gira comprometido */
13     forward = 0.0; steer = escape_dir * SPIN;
14     if (f < IR_CLEAR && fl < IR_CLEAR && fr < IR_CLEAR) {
15       pass_x = pos[0]; pass_y = pos[1]; state = CONTORNAR;
16     }
17     break;
18   case CONTORNAR: /* avança p/ passar */
19     forward = 0.7*CRUISE; steer = K_AVOID*(block_r - block_l);
20     if (front > IR_TRIG) { escape_dir = (block_l <= block_r)?+1:-1; state = GIRAR; }
21     else if (hypot(pos[0]-pass_x, pos[1]-pass_y) > PASS_DIST) state = NAVEGAR;
22     break;
23 }
24 </lstlisting>
25
26 \subsection{Parâmetros de controle}
27 A Tabela-\ref{tab:param} lista os parâmetros e os valores finais adotados (a justificativa empírica dos limiares de IR é dada na Seção-\ref{sec:calib}).

```

```

28
29 \begin{table}[H]
30 \centering
31 \caption{Parâmetros do controlador (valores finais).}
32 \label{tab:param}
33 \begin{tabular}{@{}l|l@{}}
34 \toprule
35 \textbf{Parâmetro} & \textbf{Símbolo} & \textbf{Valor} \\
36 \midrule
37 Velocidade máxima de roda & --- &  $47 \times 6 \text{ rad/s}$  \\
38 Velocidade de cruzeiro &  $v$  &  $0,25 \times 47 \times 6 \text{ rad/s}$  \\
39 Ganho de rumo &  $K_{\theta}$  &  $0$  \\
40 Ganho de desvio &  $K_a$  &  $0,05$  \\
41 Linha de base do IR &  $s_0$  &  $100$  \\
42 Limiar de disparo & \textsc{trig} &  $210$  \\
43 Limiar de liberação & \textsc{clear} &  $160$  \\
44 Distância de contorno &  $d_p$  &  $0,30 \text{ m}$  \\
45 Tolerância de chegada & --- &  $0,07 \text{ m}$  \\
46 Período do laço &  $T$  &  $16 \text{ ms}$  \\
47 \bottomrule
48 \end{tabular}
49 \end{table}
50
51 \subsection{Metodologia experimental}
52 Para obter medições reproduzíveis, o controlador foi \emph{instrumentado}: a cada segundo de
    tempo simulado registra-se uma linha de telemetria (estado, posição, distância, erro de rumo
    , leituras de IR e velocidades de roda) em arquivo, com descarga imediata de \emph{buffer}
    (\texttt{fflush}) para garantir persistência mesmo em encerramento abrupto. As execuções
    foram realizadas em modo acelerado, sem renderização (\emph{headless}), o que permitiu
    coletar centenas de ciclos de controle em poucos segundos de tempo de relógio. A análise dos
    registros norteou tanto o diagnóstico de falhas quanto a calibração dos sensores.
53
54 % =====
55 \section{Resultados e Discussão}\label{sec:resultados}
56 % =====
57
58 \subsection{Calibração empírica do sensoramento infravermelho}\label{sec:calib}
59 A folha de dados do sensor (tabela de conversão do modelo) sugere que, em espaço livre ( $>20 \text{ cm}$ 
    ), a leitura tende a  $\approx 29$ . Contudo, a telemetria coletada em espaço aberto
    evidenciou leituras de \emph{base} consistentemente em torno de  $100\text{--}120$  (Tabela-\ref{
    tab:calib}), atribuíveis à reflexão do piso captada pelos sensores infravermelhos, que são
    montados com leve inclinação. Já um obstáculo (caixa) imediatamente à frente produz leituras
     $\approx 280$ .
60
61 \begin{table}[H]
62 \centering
63 \caption{Leitura infravermelha: previsto (folha de dados) \emph{versus} medido.}
64 \label{tab:calib}
65 \begin{tabular}{@{}l|l@{}}
66 \toprule
67 \textbf{Condição} & \textbf{Previsto} & \textbf{Medido} \\
68 \midrule
69 Espaço livre ( $>20 \text{ cm}$ ) &  $\approx 29$  &  $\approx 100\text{--}120$  \\
70 Obstáculo frontal ( $\approx 3 \text{ cm}$ ) &  $\approx 150$  &  $\approx 250\text{--}300$  \\
71 \bottomrule
72 \end{tabular}
73 \end{table}
74
75 \noindent Essa discrepância tem consequência direta: limiares calibrados ``no papel'' ( $s_0=40$ ,
     $\textsc{trig}=110$ ,  $\textsc{clear}=60$ ) fazem o \emph{piso} ser interpretado como parede
    e impedem que a frente jamais ``limpe''. A recalibração para valores acima da base medida (
 $s_0=100$ ,  $\textsc{trig}=210$ ,  $\textsc{clear}=160$ ) foi decisiva. \emph{Lição:} sensores
    devem ser caracterizados por medição, não por suposição.

```

```

76 \subsection{Análise de falhas e diagnóstico}
77 \subsubsection{Aprisionamento em mínimo local}
78 A primeira versão (puramente reativa, sem compromisso) ficou \emph{presa na primeira caixa}. A
79 telemetria (Listagem~\ref{lst:stuck}) é conclusiva: a posição permanece congelada em $
(-0{,}54;-0{,}54)$, a distância estaciona em $1{,}90$-m, o erro de rumo oscila em torno de
zero ($e_{\theta} \approx 0$, pois o alvo está \emph{atrás} do obstáculo) e as rodas alternam
$-3/+3$ \to $+3/-3$ a cada ciclo --- ou seja, o robô \emph{vibra no lugar}. É a assinatura
exata do mínimo local de campo potencial.
80
81 \begin{lstlisting}[style=log,caption={Telemetria do aprisionamento em mínimo local (excerto real
).},label={lst:stuck}]
82 pos=(-0.54,-0.54) dist=1.90 err=+0.00 IR[L/F/R]=120/280/115 rodas=-3.0/+3.0
83 pos=(-0.54,-0.54) dist=1.90 err=-0.01 IR[L/F/R]=118/279/110 rodas=+3.0/-3.0
84 pos=(-0.54,-0.54) dist=1.90 err=+0.00 IR[L/F/R]=129/303/129 rodas=-3.0/+3.0
85 pos=(-0.54,-0.54) dist=1.90 err=-0.01 IR[L/F/R]=127/268/133 rodas=+3.0/-3.0

```

Listing 1: Núcleo da máquina de estados (excerto).

3.4.1 Rodopio por má calibração

Após introduzir a FSM, mas *antes* de recalibrar os sensores, surgiu uma segunda falha: o robô entrava em GIRAR já na largada e *rodopiava indefinidamente* (posição congelada em $(-0,80; -0,80)$, e_{θ} varrendo todo o círculo). A causa, identificada na Seção ??, era o limiar CLEAR estar *abaixo* da base real do piso, de modo que a condição de liberação nunca era satisfeita. Ambas as falhas foram detectadas e resolvidas exclusivamente pela leitura dos registros de telemetria.

3.5 Resultado final

Com a FSM e a calibração corretas, o robô executou a tarefa de forma **livre de colisões**. A Figura 2 mostra a trajetória percorrida sobre o mapa de obstáculos: o robô contorna a primeira caixa, atravessa o corredor central entre os obstáculos e contorna a caixa próxima ao alvo. A Figura 3 apresenta a distância ao alvo ao longo do tempo: a redução é praticamente monotônica, com um leve aumento transitório por volta de $t \in [3, 6]$ s — o “custo” geométrico da manobra de contorno da primeira caixa — até a chegada em $t = 19,3$ s.

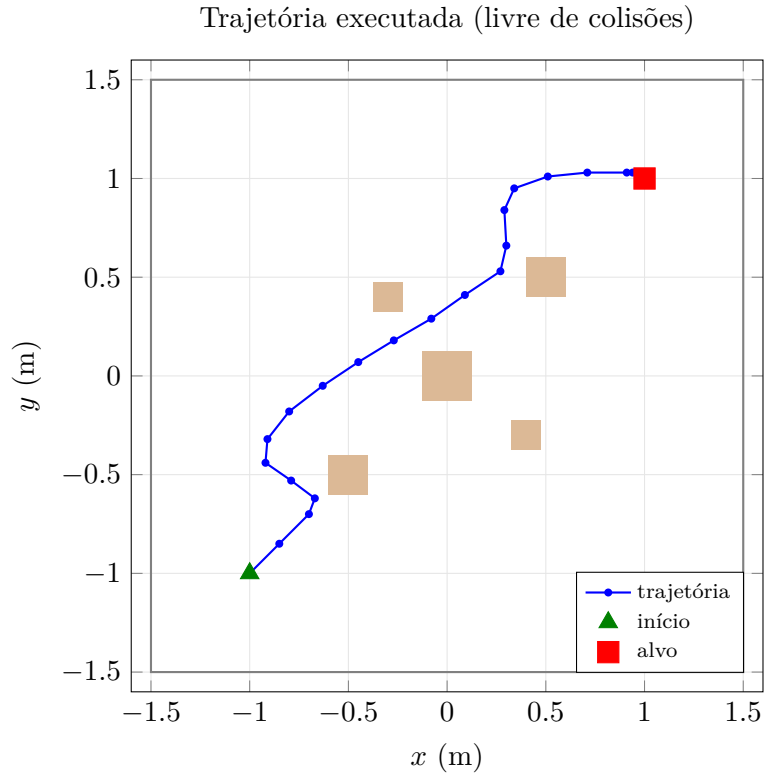


Figura 2: Trajetória do Khepera IV de $(-1, -1)$ a $(1, 1)$. Retângulos: obstáculos.

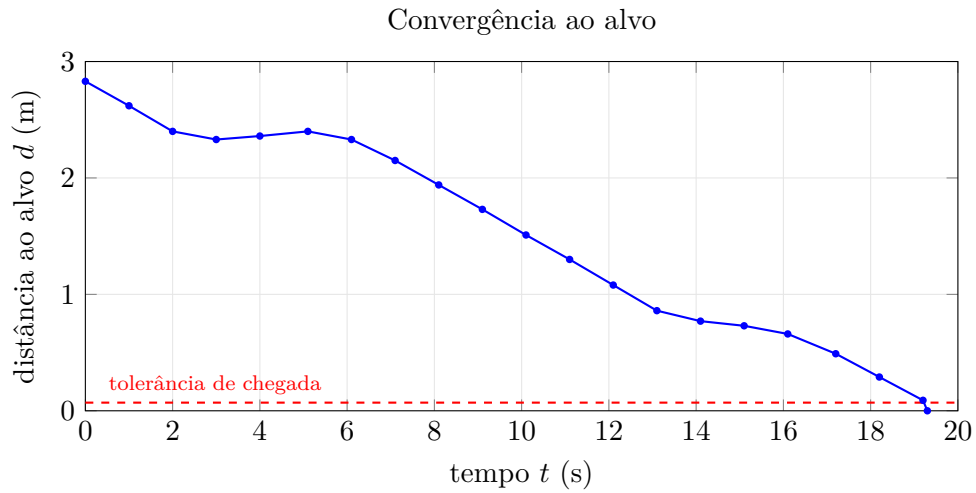


Figura 3: Distância ao alvo *versus* tempo. O leve aumento em $t \in [3, 6]$ s é o custo do contorno da primeira caixa.

A Tabela 2 sumariza o desempenho. Observou-se a sequência de estados NAVEGAR $\rightarrow$ GIRAR $\rightarrow$ CONTORNAR $\rightarrow$ NAVEGAR a cada obstáculo relevante, exatamente como projetado.

Tabela 2: Síntese do desempenho na configuração final.

Métrica	Valor
Posição inicial	(−1,00; −1,00)
Posição de chegada	(0,94; 1,03)
Distância inicial ao alvo	2,83 m
Tempo até a chegada	19,3 s (simulado)
Colisões	0
Obstáculos contornados	5 (caixas)

3.6 Discussão

Os resultados confirmam que a arquitetura híbrida com compromisso de contorno resolve a tarefa proposta de forma robusta, superando as duas falhas diagnosticadas. Três pontos merecem destaque. *Primeiro*, a importância da **observabilidade**: a telemetria instrumentada foi o instrumento decisivo de depuração — coerente com a prática de engenharia de software de tempo real, em que a inspeção do laço de controle é essencial. *Segundo*, a **calibração empírica**: o caso ilustra de modo contundente os riscos de confiar em folhas de dados sem medição. *Terceiro*, as **limitações**: (i) a abordagem reativa não é completa — geometrias adversas (p. ex. armadilhas em “U” profundas) ainda podem desafiar a heurística de escape; (ii) o uso de GPS é uma simplificação de simulação; e (iii) o curto alcance do IR impõe velocidade de cruzeiro moderada, sob pena de reação tardia (um *trade-off* de tempo real entre velocidade e distância de reação).

4 Evolução para o Robô Físico Khepera IV

Esta seção documenta a *evolução pós-entrega* do trabalho: a migração do controlador da simulação para o **Khepera IV físico** do CETELI e a implementação de vários dos itens elencados como trabalhos futuros — notadamente a troca do GPS por *odometria* (item i) e o acréscimo de uma *camada deliberativa com mapeamento e planejamento global A** (item iv). O código correspondente vive em `khepera_real/patrolha/main.c` e em uma interface web de operação (`interface/`).

4.1 Odometria e o algoritmo Bug2 no robô real

Sem GPS no robô físico, a pose (x, y, θ) passa a ser estimada por **odometria** pela integração dos encoders (147,4 pulsos/mm; bitola $L = 105,4$ mm):

$$\Delta c = \frac{1}{2}(\Delta_L + \Delta_R), \quad \Delta\theta = \frac{\Delta_R - \Delta_L}{L}, \quad x += \Delta c \cos\left(\theta + \frac{\Delta\theta}{2}\right), \quad y += \Delta c \sin\left(\theta + \frac{\Delta\theta}{2}\right), \quad (8)$$

sujeita a *deriva* acumulada, o gargalo de exatidão do sistema. A FSM da simulação foi substituída pelo **Bug2** [8] em sua forma canônica: estado GOAL (segue a linha $A \rightarrow B$), estado WALL-FOLLOW (contorna o obstáculo mantendo-o de um lado) e a *leave condition* — larga a parede assim que **recruza a linha $A \rightarrow B$ num ponto mais próximo de B** do que aquele em que a encontrou. É esse cruzamento que garante o retorno ao eixo.

4.2 Rastreamento de linha por ponto de controle virtual (ponto-P)

No controlador de simulação, o estado NAVEGAR apenas *mirava* no alvo ($\omega = K_{\theta}e_{\theta}$), ignorando o *erro lateral* à linha $A \rightarrow B$. A consequência prática, observada no robô real, é que após um contorno o robô cortava em diagonal até B em vez de *voltar ao eixo*. Adotou-se então a malha externa (cinemática) de um **controle em cascata**, na forma de um *ponto de controle virtual* P situado a uma distância d à frente do eixo das rodas.

Seja $\mathbf{u} = (u_x, u_y)$ a direção unitária $A \rightarrow B$ e $\mathbf{p} = (x - x_A, y - y_A)$. O erro lateral *signalizado* à linha é

$$d_\ell = u_x p_y - u_y p_x, \quad (9)$$

positivo quando o robô está à esquerda da linha. O rumo desejado combina o *avanço* na direção da linha com uma *correção* proporcional-integral do desvio, com um *governador* anti-windup que congela o integrador na saturação:

$$\theta_{\text{ref}} = \text{atan2}(u_y, u_x) - \text{atan2}\left(K_{ct} d_\ell + K_i \int d_\ell dt, d_\ell\right), \quad (10)$$

$$\omega = -K_\theta \text{wrap}(\theta_{\text{ref}} - \theta). \quad (11)$$

Longe da linha ($|d_\ell|$ grande) o rumo tende à *perpendicular* que reaproxima o robô do eixo; sobre a linha ($d_\ell \rightarrow 0$) tende ao *paralelo*, avançando reto para B (Figura 4). Assim, ao sair do contorno, o robô **volta ao eixo** $A \rightarrow B$ — a correção exata da re-aquisição da linha no Bug2. O ganho integral dispensa o *trim* manual de viés, e a estrutura é alternável em tempo de compilação (USE_POINT_P).

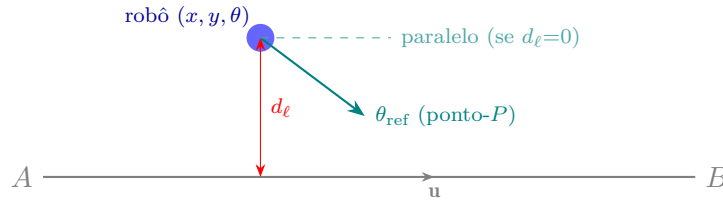


Figura 4: Rastreamento por ponto de controle virtual: o rumo desejado curva-se para reduzir o erro lateral d_ℓ e realinhar o robô à linha $A \rightarrow B$.

4.3 Camada deliberativa: mapa de ocupação e planejamento A^*

Realiza-se aqui o trabalho futuro (iv). Enquanto o robô anda, a interface constrói uma **grade de ocupação** a partir da telemetria (pose + sensores frontais), com células **free**, **visited** e **occupied**. Sobre essa grade roda um A^* 8-conexo [4] (`interface/planner.py`): as células ocupadas são *dilatadas* pelo raio do robô (espaço de configuração), o não observado é tratado como livre (otimista, pois o Bug2 reativo cobre o imprevisto), e o caminho é simplificado por *linha-de-visada* em poucos *waypoints*. O robô os percorre em um **modo missão** (`--mission`) que recebe metas `goto X Y` pela serial mantendo a *odometria contínua* entre trechos. Persiste a limitação de **referencial**: como a odometria reinicia a cada missão e deriva, mapa e alvo só são coerentes no mesmo referencial — uma localização persistente (item de trabalho futuro remanescente) seria necessária para reuso entre sessões.

4.4 Geração automática de mapa e interface de operação

Um modo de **mapeamento automático** (`--slow`) leva o robô de A a B em *meia velocidade*, contornando os obstáculos devagar para amostrar melhor, e salva o mapa ao final. Toda a operação ocorre por uma **interface web** (Flask) que conecta ao robô por USB (console serial) ou Wi-Fi/SSH, faz *deploy* e recompilação do `main.c` no próprio robô, expõe telemetria, controle manual, mapeamento e os fluxos acima. Do ponto de vista de **tempo real**, o laço de controle embarcado permanece uma tarefa periódica (período nominal de 40 ms), preservando o modelo discutido na Seção 2.

5 Conclusão e Trabalhos Futuros

Apresentou-se um controlador de navegação ponto-a-ponto com desvio de obstáculos para o Khepera IV em ambiente Webots, fundamentado na literatura clássica de robótica móvel e validado com dados reais de execução. A arquitetura híbrida arbitrada por máquina de estados finitos, aliada à calibração empírica do sensoriamento, conduziu o robô ao alvo de forma livre de colisões em 19,3 s, reduzindo a distância de 2,83 m a menos de 0,07 m. O processo de depuração baseado em telemetria mostrou-se tão instrutivo quanto o resultado, evidenciando os fenômenos de mínimo local e de descalibração sensorial. A Seção 4 documentou ainda a evolução para o **robô físico**, na qual vários dos trabalhos futuros abaixo já foram concretizados: odometria, rastreamento de linha por ponto de controle virtual e uma camada deliberativa de mapeamento com planejamento A^* .

Como trabalhos futuros, propõem-se: (i) substituir o GPS por *odometria* via encoders, aproximando-se do robô físico do CETELI e expondo o problema de deriva; (ii) incorporar os sensores ultrassônicos como “antecipação” de longo alcance, suavizando o desvio; (iii) adotar métodos reativos mais robustos como VFH [9] ou DWA [10]; e (iv) acrescentar uma camada deliberativa com mapeamento e planejamento global (A^*/RRT) para ambientes que exijam completude. Por fim, recomenda-se a análise formal de escalonabilidade do laço de controle sob o modelo de tarefas periódicas [16], fechando a ponte com a teoria de sistemas de tempo real.

Referências

- [1] W. G. Walter, “An Imitation of Life,” *Scientific American*, vol. 182, no. 5, pp. 42–45, 1950.
- [2] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. Cambridge, MA: MIT Press, 1984.
- [3] N. J. Nilsson, “Shakey the Robot,” SRI International, Technical Note 323, 1984.
- [4] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [5] R. A. Brooks, “A Robust Layered Control System for a Mobile Robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [6] R. A. Brooks, “Intelligence without representation,” *Artificial Intelligence*, vol. 47, no. 1–3, pp. 139–159, 1991.
- [7] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [8] V. J. Lumelsky and A. A. Stepanov, “Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape,” *Algorithmica*, vol. 2, pp. 403–430, 1987.
- [9] J. Borenstein and Y. Koren, “The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots,” *IEEE Trans. Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.
- [10] D. Fox, W. Burgard, and S. Thrun, “The Dynamic Window Approach to Collision Avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [11] F. Mondada, E. Franzi, and P. Ienne, “Mobile robot miniaturisation: A tool for investigation in control algorithms,” in *Proc. 3rd Int. Symp. Experimental Robotics (ISER)*, 1993, pp. 501–513.

- [12] J. M. Soares, I. Navarro, and A. Martinoli, “The Khepera IV Mobile Robot: Performance Evaluation, Sensory Data and Software Toolbox,” in *Robot 2015: Second Iberian Robotics Conf.*, Springer, 2016, pp. 767–781.
- [13] O. Michel, “Cyberbotics Ltd. — WebotsTM: Professional Mobile Robot Simulation,” *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.
- [14] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [15] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [16] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

A Código-fonte completo do controlador

A listagem a seguir reproduz, na íntegra, o controlador `goto_avoid.c`.

```

1  /*
2  * goto_avoid.c -Khepera IV: navegacao ponto-a-ponto com desvio de obstaculos
3  * -----
4  * Disciplina: Engenharia de Software de Tempo Real (CETELI/UFAM)
5  *
6  * v2 -MAQUINA DE ESTADOS (corrige o "minimo local" da v1).
7  *
8  * Campo potencial puro TRAVA quando o alvo fica atras de um obstaculo
9  * (a forca que puxa anula a que empurra). A cura classica e PARAR de ser
10 * 100% reativo e se COMPROMETER com um contorno (ideia dos algoritmos Bug
11 * e do wall-following). Por isso 3 estados:
12 *
13 * NAVEGAR -> vai pro alvo (controle P de rumo) + desvio reativo leve
14 * GIRAR -> parede na frente: trava 1 direcao e gira ate limpar
15 * CONTORNAR-> anda reto uma distancia fixa p/ passar a caixa, e re-mira
16 *
17 * Tudo num laco periodico (wb_robot_step) = tarefa periodica de tempo real.
18 */
19
20 #include <webots/robot.h>
21 #include <webots/motor.h>
22 #include <webots/distance_sensor.h>
23 #include <webots/gps.h>
24 #include <webots/inertial_unit.h>
25
26 #include <stdio.h>
27 #include <math.h>
28
29 #define MAX_SPEED 47.6 /* rad/s -velocidade max das rodas */
30 #define CRUISE (0.25 * MAX_SPEED) /* velocidade de cruzeiro (~0.25 m/s) */
31
32 #define GOAL_X 1.0 /* ponto B (bate com o disco vermelho) */
33 #define GOAL_Y 1.0
34 #define GOAL_TOL 0.07 /* 7 cm: "chegou" */
35
36 /* limiares calibrados pela MEDICAO real (piso reflete -> base -90-120): */
37 #define IR_BASE 100.0 /* leitura de base em espaco aberto */
38 #define IR_TRIG 210.0 /* >210: obstaculo de verdade -> GIRAR */
39 #define IR_CLEAR 160.0 /* <160: frente limpou (volta p/ base) */
40
41 #define K_HEADING 3.0 /* ganho P do rumo (go-to-goal) */

```

```

42 #define K_AVOID 0.05 /* ganho do desvio reativo */
43 #define SPIN 6.0 /* rad/s p/ girar no lugar no estado GIRAR */
44 #define PASS_DIST 0.30 /* m a percorrer no estado CONTORNAR */
45
46 typedef enum { NAVEGAR, GIRAR, CONTORNAR } State;
47
48 static double clampd(double v, double lo, double hi) {
49     return v < lo ? lo : (v > hi ? hi : v);
50 }
51 static double wrap_pi(double a) {
52     while (a > M_PI) a -= 2.0 * M_PI;
53     while (a < -M_PI) a += 2.0 * M_PI;
54     return a;
55 }
56 static double maxd(double a, double b) { return a > b ? a : b; }
57
58 int main(void) {
59     wb_robot_init();
60     setvbuf(stdout, NULL, _IONBF, 0); /* sem buffer: telemetria sai na hora */
61     FILE *logf = fopen("U:\\ufam\\eng_soft_rt\\controllers\\goto_avoid\\run.log", "w");
62     const int dt = (int)wb_robot_get_basic_time_step();
63
64     /* --- atuadores: rodas em modo VELOCIDADE --- */
65     WbDeviceTag left = wb_robot_get_device("left wheel motor");
66     WbDeviceTag right = wb_robot_get_device("right wheel motor");
67     wb_motor_set_position(left, INFINITY);
68     wb_motor_set_position(right, INFINITY);
69     wb_motor_set_velocity(left, 0.0);
70     wb_motor_set_velocity(right, 0.0);
71
72     /* --- sensores IR do anel --- */
73     const char *ir_names[8] = {
74         "rear left infrared sensor", "left infrared sensor",
75         "front left infrared sensor", "front infrared sensor",
76         "front right infrared sensor", "right infrared sensor",
77         "rear right infrared sensor", "rear infrared sensor"};
78     WbDeviceTag ir[8];
79     for (int i = 0; i < 8; i++) {
80         ir[i] = wb_robot_get_device(ir_names[i]);
81         wb_distance_sensor_enable(ir[i], dt);
82     }
83
84     /* --- localizacao: GPS + IMU --- */
85     WbDeviceTag gps = wb_robot_get_device("gps");
86     wb_gps_enable(gps, dt);
87     WbDeviceTag imu = wb_robot_get_device("inertial unit");
88     wb_inertial_unit_enable(imu, dt);
89
90     State state = NAVEGAR;
91     int escape_dir = 1; /* +1 = gira p/ esquerda, -1 = p/ direita */
92     double pass_x = 0, pass_y = 0;
93     const char *snames[3] = {"NAV", "GIR", "CON"};
94
95     printf("[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
96     if (logf) { fprintf(logf, "[goto_avoid v2] objetivo: ir ate (%.2f, %.2f)\n", GOAL_X, GOAL_Y);
97         fflush(logf); }
98     double last_print = -1.0;
99
100     /* ===== LACO DE CONTROLE ===== */
101     while (wb_robot_step(dt) != -1) {
102         /* ---- (1) PERCEPCAO ---- */
103         const double *pos = wb_gps_get_values(gps);

```

```

104 double yaw = wb_inertial_unit_get_roll_pitch_yaw(imu)[2];
105 double dx = GOAL_X - pos[0], dy = GOAL_Y - pos[1];
106 double dist = sqrt(dx * dx + dy * dy);
107
108 if (dist < GOAL_TOL) { /* chegou -> para tudo */
109     wb_motor_set_velocity(left, 0.0);
110     wb_motor_set_velocity(right, 0.0);
111     printf(">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1], wb_robot_get_time());
112     if (logf) { fprintf(logf, ">> CHEGUEI em (%.2f, %.2f) | t = %.1f s\n", pos[0], pos[1],
113         wb_robot_get_time()); fflush(logf); fclose(logf); logf = NULL; }
113     break;
114 }
115
116 double f = wb_distance_sensor_get_value(ir[3]); /* frente */
117 double fl = wb_distance_sensor_get_value(ir[2]); /* frente-esq */
118 double fr = wb_distance_sensor_get_value(ir[4]); /* frente-dir */
119 double l = wb_distance_sensor_get_value(ir[1]); /* esquerda */
120 double r = wb_distance_sensor_get_value(ir[5]); /* direita */
121 double block_l = clampd((fl - IR_BASE) + 0.5 * (l - IR_BASE), 0, 1e9);
122 double block_r = clampd((fr - IR_BASE) + 0.5 * (r - IR_BASE), 0, 1e9);
123 double front = maxd(f, maxd(fl, fr)); /* "tem parede a frente?" */
124
125 double goal_dir = atan2(dy, dx);
126 double err = wrap_pi(goal_dir - yaw);
127
128 /* ---- (2) MAQUINA DE ESTADOS ---- */
129 double forward = 0.0, steer = 0.0;
130 switch (state) {
131
132     case NAVEGAR:
133         /* vai pro alvo (P de rumo) + desvio reativo leve */
134         steer = K_HEADING * err + K_AVOID * (block_r - block_l);
135         forward = CRUISE;
136         forward *= (1.0 - 0.7 * clampd((f - IR_BASE) / (IR_TRIG - IR_BASE), 0, 1));
137         forward *= clampd(1.0 - 0.5 * fabs(err), 0.3, 1.0);
138         if (front > IR_TRIG) { /* parede! decide UMA vez e trava */
139             escape_dir = (block_l <= block_r) ? +1 : -1; /* gira p/ o lado livre */
140             state = GIRAR;
141         }
142         break;
143
144     case GIRAR:
145         /* gira no lugar, MESMA direcao, ate a frente limpar de verdade */
146         forward = 0.0;
147         steer = escape_dir * SPIN;
148         if (f < IR_CLEAR && fl < IR_CLEAR && fr < IR_CLEAR) {
149             pass_x = pos[0];
150             pass_y = pos[1];
151             state = CONTORNAR;
152         }
153         break;
154
155     case CONTORNAR:
156         /* anda reto p/ passar a caixa; so desvio leve; re-gira se topa outra */
157         forward = 0.7 * CRUISE;
158         steer = K_AVOID * (block_r - block_l);
159         if (front > IR_TRIG) {
160             escape_dir = (block_l <= block_r) ? +1 : -1;
161             state = GIRAR;
162         } else {
163             double tx = pos[0] - pass_x, ty = pos[1] - pass_y;
164             if (sqrt(tx * tx + ty * ty) > PASS_DIST)
165                 state = NAVEGAR; /* contornou -> re-mira no alvo */

```

```

166     }
167     break;
168 }
169
170 /* ---- (3) CINEMATICA DIFERENCIAL ---- */
171 double vl = clampd(forward - steer, -MAX_SPEED, MAX_SPEED);
172 double vr = clampd(forward + steer, -MAX_SPEED, MAX_SPEED);
173 wb_motor_set_velocity(left, vl);
174 wb_motor_set_velocity(right, vr);
175
176 /* ---- telemetria (1x/s) ---- */
177 double t = wb_robot_get_time();
178 if (t - last_print >= 1.0) {
179     last_print = t;
180     printf("[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]=%3.0f/%3.0f/%3.0f
181           rodas=%.1f/%.1f\n",
182           snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr);
183     if (logf) { fprintf(logf, "[%s] t=%4.1fs pos=(%.2f,%.2f) dist=%.2f err=%.2f IR[L/F/R]
184           ]=%3.0f/%3.0f/%3.0f rodas=%.1f/%.1f\n",
185           snames[state], t, pos[0], pos[1], dist, err, fl, f, fr, vl, vr); fflush(logf); }
186 }
187 }
188
189 wb_robot_cleanup();
190 return 0;
191 }

```